

TP n° 3 : Processus, premiers scripts BASH

Administration Système - ESIEE E3INFO - 2025

Objectifs du TP : Connaître les commandes de la gestion des processus, les redirections et programmer des scripts avec le langage BASH.

Rendu : Le fichier est à compléter (partie 1, 2 et 5), les scripts doivent être écrits dans des fichiers séparés. LE tout est à rendre sur elearning dans une archive.

1 Gestion des processus

1. Lancez votre machine virtuelle sous Linux
2. Avec l'éditeur « vim » ou « nano », saisir ce programme, nommé « pg_erreur », dans le répertoire courant.

```
#!/bin/bash
while (true)
do
    echo Bonjour
done
```

3. Enregistrez-le.
4. Rendez-le fichier « pg_erreur » exécutable.
5. Exécutez-le.
6. Que se passe-t-il ?
Le terminal affiche que des bonjours.
7. Arrêtez le programme par Ctrl-C.
8. Relancez le programme mais en faisant suivre son nom par un « & ».
9. Arrêtez le programme par Ctrl-C. Que constatez-vous ?
Qu'on ne peut pas arrêter le programme.
10. Pourquoi ?
Car le programme est exécuté en arrière plan.
11. Allez sur TTY2 et connectez-vous (Ctrl Droit + F2)
12. La commande « ps » liste tous les processus actifs de la session utilisateur. Quelle commande doit-t-on taper pour voir tous les processus en cours ?
On doit taper ps aux.
13. Recherchez le script que vous venez d'écrire. Comment le reconnaissez-vous ?
Je le reconnais avec la commande ps aux | grep pg_erreur.
14. Quel est son identifiant ?
Son PID est 3795290.
15. Quel est l'identifiant de son père ?
Son PPID est 4110131.

```
(debian-trixie-2025)tpreseau@pccop0b230-11:~$ ps -p 3795290 -f
UID          PID    PPID  C STIME TTY          TIME CMD
tpreseau 3795290 4110131 37 09:17 pts/4    00:01:29 /bin/bash ./pg_erreur.sh
```

16. Lancer la commande « top ». Où est situé le programme « pg-erreur » ?
Il est situé en première ligne.
17. Quel est le processus qui consomme le plus de temps processeur ?
C'est pg_erreur.
18. Quittez « top » en appuyant sur « q ».
19. Tuez le processus de « pg-erreur » à l'aide de la commande « kill » (ou avec « top »).
20. Vérifiez avec « top » que tout est rentré dans l'ordre. Comment le voyez-vous ?
On ne voit plus pg_erreur dans top.
21. Relancez à nouveau le script précédent, et interrompez-le avec la commande Ctrl-Z, que se passe-t'il ?
Le script est arrêté.
22. Tapez la commande « fg », que se passe-t'il ?
Il relance le script arrêté.
23. Tapez à nouveau Ctrl-Z, puis tapez la commande « bg », que se passe-t'il ? Et comment se sortir de cette situation ?
Le script est lancé en arrière plan. Pour sortir de cette situation, il faut lancer un nouveau terminal et taper la commande kill + PID de pg_erreur pour l'arrêter.
24. Expliquer brièvement le fonctionnement des commandes « nice » et « nohup »
nice - Exécuter un programme avec une politesse modifiée.
nohup - Exécuter une commande en la rendant insensible aux déconnexions, avec une sortie hors terminal.

2 Redirections

1. Trouver une commande afin de créer un fichier `/home/user/fichiers` (user est votre nom d'utilisateur) contenant les noms des fichiers contenus dans le répertoire `/var/log`
`ls /var/log > fichier`
2. Trouver une commande qui ajoute les noms des fichiers du répertoire `/etc` au fichier créé à la question précédente.
`ls /etc >> fichier`
3. Trouver une commande qui crée un deuxième fichier `/home/user/fichiers_trie` résultat du tri du premier fichier (la commande `sort` permet de trier les lignes d'un fichier).
`Sort fichier > fichier_trie`
4. Avec l'aide de `cat`, trouver une commande qui permet de concaténer les deux fichiers précédents dans un seul fichier nommé `tout`
`cat fichier fichier_trie > tout`
5. Trouver une commande qui compte le nombre de processus lancés sur votre machine.
`Ps aux | wc -l`

3 Quelques scripts simples

1. Écrivez et testez les exercices du cours sur Bash (inutile de les rendre)
2. Ecrivez un script nommé *deuxfois* qui affiche le message "Entrez un mot : ", lit le mot saisi par l'utilisateur puis affiche ce mot deux fois sur la même ligne.
 Exemple: \$ **./deuxfois**
 Entrez un mot : **toto**
 toto toto
 \$
3. Ecrire un script nommé *untrois* qui demande à l'utilisateur de saisir une suite de mots constituée d'au moins trois mots et qui affiche sur la même ligne le premier et le troisième mot saisis.
 Exemple : \$ **./untrois**
 Entrez une suite de mots : **un petit coucou de Pornichet**
 un coucou
 \$
4. Écrire les expressions génériques qui correspondent aux motifs suivants :
 1. Les mots qui commencent par un caractère quelconque, et qui se terminent par un h
`*h$`
 2. Les mots qui contiennent la chaîne ftp mais qui ne commencent pas par un f
`^[^f]*ftp*`
 3. Les mots qui contiennent deux fois au moins une des lettres parmi A, B ou C
`.*(A.*A|B.*B|C.*C).*`
 4. Les mots de 4 caractères qui commencent par une lettre majuscule.
`^[A-Z][a-zA-Z]{3}$`
5. Écrivez un script qui demande à l'utilisateur de saisir un nombre de bits et qui affiche un message d'erreur si cette valeur est inférieure à 0, ou alors la conversion de cette valeur en nombre d'octets.

Modifiez votre script, et faites en sorte que le programme se répète tant que l'utilisateur n'a pas saisi la touche q. A la fin de l'exécution le script doit afficher la somme totale des valeurs saisies en nombre d'octets.

Puis faites en sorte que la valeur saisie puisse être donnée en paramètre au programme, ou bien être saisie si elle n'a pas été trouvée en paramètre.

Voir le script [bits.sh](#)

6. *Rappel*: Les variables \$1, \$2, \$3... désigne les valeurs des arguments passés à un script.

En utilisant une structure *case*, écrire un script nommé *3car* qui prend en argument une chaîne de caractères et qui affiche OK si la chaîne est composé de 3 caractères seulement. Si il y en a plus il affiche un message d'erreur (code de retour 2), sinon il n'affiche rien. Vous vérifierez également qu'il n'y a qu'un seul paramètre.

Voir le script [3car.sh](#)

4 Réponse en temps limité (source : X. Hilaire)

La commande interne `read` du shell permet de lire une ligne à partir du terminal, mais attend indéfiniment que l'utilisateur veuille bien en saisir une. Dans cet exercice, on cherche à écrire un script qui attend la réponse de l'utilisateur en un temps limité : s'il ne saisit rien au-delà d'une limite de temps, le script doit supposer une réponse par défaut, et poursuivre son exécution.

1. Écrire un premier script `question.sh` qui fera les choses suivantes :
 - Il affiche son PID sur sa sortie d'erreur standard
 - Il affiche son unique argument, supposé être une question
 - Il lit la réponse de l'utilisateur, qu'on supposera de type oui/non
 - Si l'utilisateur a répondu O ou o, le script doit terminer avec le code de sortie 0, sinon avec 1.

Exemple de résultat attendu :

```
clement@CAMANA2:TP4$ ./question.sh "Aimez-vous Bash ? (O/N)"
PID = 4844
Aimez-vous Bash ? (O/N)O
clement@CAMANA2:TP4$ echo $?
0
clement@CAMANA2:TP4$ ./question.sh "Aimez-vous Bash ? (O/N)"
PID = 4921
Aimez-vous Bash ? (O/N)Non
clement@CAMANA2:TP4$ echo $?
1
```

2. Modifiez le script précédent pour garantir qu'il termine au bout de 5 secondes, même si l'utilisateur ne saisit rien. Vous pourrez utiliser un autre script ou processus, et les commandes `kill` et `sleep`, vues en cours, pour ce faire.

Exemple de résultat attendu :

```

clement@CAMANA2:TP4$ date;./question2.sh "Aimez-vous Bash ? (O/N)"
Tue Oct  7 21:36:11 CEST 2025
PID = 4554
Aimez-vous Bash ? (O/N)
clement@CAMANA2:TP4$ date
Tue Oct  7 21:36:25 CEST 2025
clement@CAMANA2:TP4$ 

```

- Examinez le code de sortie retourné par le shell dans ce dernier cas de figure où l'utilisateur ne répond pas à la question

Le code de sortie est 143 car le processus est kill par un signal.

5 Recherches avec grep

La fonction grep permet de faire des recherches dans les lignes d'un fichier ou d'un flux à partir d'une expression régulière (avec l'option -e)

Vous êtes invités à lire le manuel pour la réalisation de cet exercice afin de répondre aux questions suivantes (une ligne de commande suffit pour y répondre) :

- A combien de groupes appartient votre utilisateur sur votre machine ?

`groups | grep $USER | wc -w`

- Combien il y a-t-il de fichiers exécutables par tout le monde dans le répertoire /usr/bin ?

`ls -l /usr/bin | grep -e '-..x..x..' | wc -l`

- Combien de script bash utilisant avez-vous écrit pour ce cours (contenant la chaîne d'appel # !/bin/bash » ? Afficher également les noms des fichiers

`"grep -rl -e '^#!/bin/bash' ."` => pour afficher les noms

`"grep -rl -e '^#!/bin/bash' . | wc -l"` => pour afficher le nombre

- Combien y a-t-il de lignes vides dans la documentation de bash ?

`man bash | grep -c -e '^$'`

- Afficher les numéro de tous les processus (PID) de votre utilisateur courant

`ps -au | grep $USER | awk '{print $2}'`